

AsymptoticAnalysis

Incremental source and runtime audit

Condition propagation across Wolfram 15 and Mathics3

Prepared for	Vladimir Reshetnikov
Audit date	September 10, 2026
Repository	https://github.com/VladimirReshetnikov/Asymptotic
Pinned commit	<code>ef1aeec4845a9c35e140963a0333d0c9ec33b05</code>
Exclusion baseline	Six waves under <code>external-reports/code-review</code> ; maintained finding register and selected predecessor articles
Executed kernel	Wolfram Language 15.0.0, Linux x86 (64-bit), build dated May 6, 2026
Mathics evidence	Shared-source analysis; no Mathics execution in this audit

Principal result. A fresh operational-condition defect is reproduced with exact polynomial input: repeatedly adding zero creates exponentially many copies of the same domain predicate while the displayed mathematical answer remains unchanged. A two-site repair mechanism is exercised in Wolfram 15, including preservation of distinct parameter assumptions and rejection of incompatible domains.

What the package contains. This article, guarded patch-staging tools, focused Wolfram Language probes, independent Python tests, source fixtures, an evidence ledger, and a novelty crosswalk. The complete supplied probe scripts and full staged integration have not been run. No full upstream-suite or Mathics acceptance result is claimed.

Abstract

This is an incremental review rather than another copy of the repository's extensive existing backlog. Inspection covered central construction, arithmetic, inverse, special-function, numerical-evidence and compatibility paths, with focused execution on the pinned standalone package. One new root cause is admitted to the findings register: live branch and assumption predicates are duplicated during ordinary binary arithmetic. The alignment stage already combines both operands' conditions, but the final result construction combines the left operand again. With a fixed independently constructed zero series, the number of copies of a single domain predicate is $2^{n+1} - 1$ after n additions. With the ordinary scalar zero, operand promotion inherits the current conditions and the observed recurrence becomes 3^n .

The distinction from previously reported recipe/provenance growth matters: this defect is visible in the current `TargetDomain` alone, before serialization or refinement. The most obvious one-line repair is incomplete; a native test shows that scalar growth merely changes from 3^n to 2^n . A stronger candidate combines duplicate-free structural conjunction with single ownership of the aligned condition set. Selected native tests retain one predicate throughout and preserve branch restrictions. The accompanying independent tests establish the count model, Boolean equivalence and guarded source transformation, not interpreter correctness. The article details the source path, mathematical invariant, repair boundary, portability obligations and precise remaining acceptance work.

Contents

1	Conclusions and scope	2
2	The representation boundary that fails	3
3	Native witnesses and exact growth laws	5
4	A repair that is sufficient for the demonstrated mechanism	7
5	Distinction from the existing review backlog	9
6	Controls and the boundaries of the evidence	9
7	Wolfram 15 and Mathics3: targeted portability analysis	10
8	Focused implementation and development plan	11
9	Reproducing and using the bundle	12
10	Final assessment	13

1 Conclusions and scope

1.1 The new finding

The actionable finding is **CG-01: exponential amplification of operational proof conditions in binary series arithmetic**. Its primary classification is performance and representation robustness, not a newly demonstrated wrong asymptotic coefficient. The exact witness returns the correct expression x at every measured step. The problem is that a mathematically idempotent condition intersection is implemented as a repeatedly enlarged expression tree. The responsible sites are in `src/Kernel/SeriesOperations.wl`, particularly lines 129–150; scalar promotion is in `src/Kernel/SeriesArithmetic.wl`, lines 49–92.[2, 3]

Assessment	Conclusion
Priority	Fix before expanding arithmetic-heavy workflows. Even neutral operations amplify data subsequently used by symbolic proof and coefficient-validation paths.
Reproduction	Successful public-package observations in Wolfram 15.0.0.
Native repair evidence	Two exact source substitutions; three four-step chains, a distinct-parameter control, and a contradictory-domain control.
Mathics status	The same shared source is implicated. Actual Mathics predicate counts, candidate execution and performance remain unverified.
Scope of remedy	Operational conjunction construction only. No recipe graph, coefficient algebra, cache architecture or branch policy is replaced.

I do not count the fixed-zero case, scalar-zero case, multiplication case and insufficient first repair as four defects. They are consequences and controls of one condition-propagation defect. Likewise, the recurrence proof, native count table and Python model are distinct forms of evidence for that same defect, not independent package acceptance totals.

1.2 Why the review is pinned

The reviewed revision is `efalaeec4845a9c35e140963a0333d0c9ec33b05`. Its commit records wave-6 review intake and the scope of previously verified option spelling. Findings here concern that snapshot; they do not assert that a later checkout remains unchanged.[1]

The exclusion baseline is unusually substantial. The six review indexes and maintained status register were screened, and selected predecessor articles and source passages were inspected. The maintained register distinguishes the 231 attributed entries through wave 4 from the later waves, which lack consolidated intake. An attributed entry is not necessarily a distinct current defect. The present article does not reproduce that backlog or treat its historical observations as fresh tests.[4, 5, 6, 7]

The novelty check is bounded rather than absolute. It is not a claim that every line of all retained articles, source files, tests and notebooks was reread. In particular, an empty GitHub search response was not accepted as proof of absence: a search for a known source symbol also returned no matches. The supplied novelty ledger states which neighboring findings were considered and why CG-01 is more specific than those entries.

1.3 Inspection coverage

The review followed execution paths rather than merely collecting names of unsupported functions. The following table describes substantive coverage; it is not a certification of any module.

Area	Inspected implementation	Evidence obtained
Public construction and dispatch	Selected main-kernel entry points, result representation, residual and numerical dispatch	Source path tracing; successful package loading and public calls
Ordinary arithmetic and calculus	SeriesOperations, SeriesArithmetic, observable application, coordinate flattening	New condition-growth witness; 41 eligible baseline observable comparisons
Inverse and transformed coordinates	Ordinary numerical evidence, CoordinateInverse, logarithmic and reciprocal-log operations	Exact-equation and retained-source-domain controls; source analysis
Gamma and Barnes families	Gamma construction and powers, Barnes coefficient construction, special adapters	Parameter, observable and recurrence inspection; no fresh coefficient defect admitted
Special-function normalization	Exact identities, parameterized functions, selected native-tree transport	Branch and remainder reasoning; no speculative private-helper issue promoted as a public defect
Certificates	Selected interval and accuracy logic plus numerical-evidence boundaries	Comparison against the exclusion baseline; no old certificate issue repeated
Mathics compatibility	Assumptions, Taylor, basic compatibility, numerical, algebra and time-budget adapters	Source-level semantic analysis only; no local Mathics installation
Prior review corpus	All six wave indexes, maintained register, selected full articles and targeted predecessor passages	Exclusion screening and CG-01 crosswalk

The full upstream suite was not run. Neither the selection of source modules nor the success of small controls establishes complete Wolfram/Mathics parity. `evidence/source_coverage.json` gives the more detailed file-level coverage distinction.

2 The representation boundary that fails

2.1 A series object carries more than its printed expression

A derived ordinary result stores a finite expression, a remainder, an internal series representation, assumptions, a target domain and a refinement recipe. In `seriesMake`, the current representation's `Domain` is copied into the public `TargetDomain`; the representation itself is retained separately. The same function performs real-coefficient validation using the current proof environment. These facts make the size of operational conditions relevant even when the expression and coefficient support are tiny.[2, lines 100–121]

For the witness, the mathematical data are deliberately uninteresting:

$$F(x) = x, \quad Z(x) = 0, \quad x \rightarrow 0^+.$$

Both are exact. Adding Z neither increases the number of nonzero coefficient blocks nor creates an unknown tail. The initial target condition is simply $D = (x > 0)$. Counting copies of D in `TargetDomain` therefore isolates a bookkeeping failure rather than a difficult asymptotic calculation.

The arithmetic path is:

```
SeriesAdd[s, t]
-> seriesArithmeticPublicBinary["Add", ...]
-> seriesArithmeticBinary["Add", ...]
-> seriesRegularOperand[t, s, ...] (* only for regular t *)
-> seriesBinary["Add", s, operand, ...]
-> seriesMake[representation, recipe, ...]
```

The public layer validates the cutoff and controls automatic arithmetic; the inner dispatcher admits analytic objects, handles other scale families, promotes ordinary operands, and enters the ordinary binary engine. Those layers explain why an explicit call with scalar 0 is not the same construction path as a call with an independently created zero series.[3, lines 35–92]

2.2 Alignment already intersects both condition sets

After validating compatible variables, scales and branch domains, `seriesBinary` aligns the right representation with the left logarithmic variable:

```
a = seriesData[s, limit];
b = seriesData[t, limit];
seriesCompatible[a, b];
b = seriesAlign[a, b];
```

The alignment definition is:

```
seriesAlign[a_, b_] := Join[
  b /. b["LogVariable"] -> a["LogVariable"],
  <|"Assumptions" -> a["Assumptions"] && b["Assumptions"],
  "Domain" -> Lookup[a, "Domain", True] &&
    Lookup[b, "Domain", True]|>];
```

Thus, if the incoming condition expressions are A and B , the local variable b now owns $A \wedge B$. It no longer contains only the original right operand's conditions. That ownership change is the key fact a reader must track.[2, lines 123–135]

The subsequent optional flattening does not undo this change. `seriesFlat` either returns the existing representation or replaces only its prefactor, offset and jet. Its domain and assumptions remain attached. Consequently the final join still sees the aligned condition set in b . [2, lines 93–99]

2.3 The final merge adds the left conditions again

Near the end of `seriesBinary`, the result is assembled with:

```
d = Join[d, <|
  "Assumptions" -> a["Assumptions"] && b["Assumptions"],
  "Domain" -> Lookup[a, "Domain", True] &&
    Lookup[b, "Domain", True],
  "RemainderDerivativeOrder" -> order|>];
```

Because b is already aligned, this persists

$$A \wedge (A \wedge B)$$

rather than one representation of $A \wedge B$. This is logically correct: conjunction is idempotent. It is not structurally economical. The native evaluator retains repeated symbolic predicates instead of automatically performing the needed idempotence reduction. A simple native control returned $a > 0 \ \&\& \ a > 0$ unchanged.[2, lines 146–150]

There is also a temporary proof environment built from `seriesAss[a]` and `seriesAss[b]`. The supplied candidate deliberately leaves that expression unchanged: the persistent condition defect can be addressed without simultaneously redesigning all proof-call construction. This separation limits the repair's scope and makes its correctness argument inspectable.

3 Native witnesses and exact growth laws

3.1 Fixed independent zero series

Evidence status: Executed on the unchanged pinned package in Wolfram 15.0.0.

The essential reproduction is:

```
s = AsymptoticAnalysis'AsymptoticExpansion[
  x, {x, 0, 3}, "Backend" -> "Package"];
z = AsymptoticAnalysis'AsymptoticExpansion[
  0, {x, 0, 3}, "Backend" -> "Package"];

Table[
  s = AsymptoticAnalysis'SeriesAdd[s, z];
  {Normal[s],
   Count[s["TargetDomain"], HoldPattern[x > 0], {0, Infinity}]}],
  {8}]
```

The expression remains x . The full measurement, including the initial state, was:

Step	Copies of $x > 0$	Domain LeafCount	Object ByteCount	Step time (s)
0	1	3	4,584	—
1	3	10	15,968	0.017234
2	7	22	28,464	0.003814
3	15	46	43,088	0.005528
4	31	94	60,112	0.006444
5	63	190	81,744	0.008595
6	127	382	113,936	0.010146
7	255	766	168,144	0.013230
8	511	1,534	266,384	0.027537

These are one-session observations, not a benchmark study. `ByteCount` is the value reported for the whole object, not peak resident memory or a measurement that separates runtime sharing. The timings are unreplicated and do not establish a speedup or an asymptotic runtime law. The strongest observation is the deterministic predicate count in the current domain alone.

Proposition 3.1 (Fixed-right-operand growth). *Suppose the incoming left domain contains d_n copies of one atomic predicate and the fixed right domain contains one copy. If the two joins above retain those occurrences, then*

$$d_{n+1} = 2d_n + 1, \quad d_0 = 1, \quad \text{hence} \quad d_n = 2^{n+1} - 1.$$

Proof. Alignment contributes the d_n left occurrences and one right occurrence to `b`. The final merge adds the d_n left occurrences a second time. Solving the affine recurrence, or inducting directly, gives the formula. $\square$

For the measured nontrivial conjunctions, the domain leaf count is $1 + 3d_n$: one conjunction head plus d_n copies of a three-leaf comparison. The initial non-conjunction state has leaf count three. Both structural predictions match the table exactly.

3.2 Scalar zero is worse

Evidence status: Executed on the unchanged pinned package in Wolfram 15.0.0.

Replacing the independently constructed z by scalar 0 produced:

```
(* Repeated SeriesAdd[s, 0] *)
{{x, 3}, {x, 9}, {x, 27}, {x, 81}}
```

This is not contradictory evidence. `seriesRegularOperand` starts with `d = seriesData[s, limit]`, replaces the coefficient/offset/prefactor fields for the regular operand, and calls `seriesMake` while retaining `d`'s domain and assumptions. The promoted scalar therefore inherits all d_n occurrences from the current left operand.[3, lines 49–69]

Proposition 3.2 (Inherited-scalar growth). *For a promoted scalar operand whose condition expression is inherited from the left representation, the same implementation gives*

$$d_{n+1} = 3d_n, \quad d_0 = 1, \quad \text{hence} \quad d_n = 3^n.$$

Proof. The promoted right operand contains d_n occurrences. Alignment constructs $2d_n$ occurrences in `b`; the final result adds the original d_n once again. $\square$

The same source path is used for ordinary scalar multiplication. The complete candidate was tested on repeated multiplication by one; the unchanged baseline scalar-multiplication count is a source-derived prediction in the supplied runner, not an additional measured baseline result.

3.3 Operation-order sensitivity

For a fixed independently constructed left zero operand and a growing right operand, the source model predicts $d_{n+1} = d_n + 2$, hence $d_n = 2n + 1$. In contrast, the measured fixed-right chain has $2^{n+1} - 1$ occurrences. Consequently two mathematically equivalent ways to accumulate additions can have radically different condition-tree sizes. This particular commuted native chain was not measured; its count is an independently tested structural consequence of the source rule.

This is a useful acceptance dimension: verify equality of mathematical results and equivalence of domains, but also check that operand order does not introduce avoidable multiplicity growth. It does not follow that the entire result associations should be `SameQ`; recipes, stable predicate order and chosen representation may legitimately differ.

3.4 Consequences without overstating them

The observed defect can burden later real-coefficient checks, domain compatibility checks, printing, traversal and result manipulation. Those operations receive the enlarged conditions through the representation. A high-level caller can see a one-term exact answer while carrying hundreds of repetitions of the same restriction.[2]

The observations do not demonstrate that a wrong branch was accepted, that a coefficient became false, or that every arithmetic chain reaches a timeout. They do show unnecessary exponential structural work in an ordinary public path. Increasing `MaxTerms` or using a faster proof engine would not remove that construction error. Likewise, suppressing domain checking would be an unacceptable workaround: the restrictions remain necessary even when their repetitions are not.

4 A repair that is sufficient for the demonstrated mechanism

4.1 The first repair is instructive but incomplete

Since b already owns the combined conditions, the first candidate changed only the final persistent merge:

```
d = Join[d, <|
  "Assumptions" -> b["Assumptions"],
  "Domain" -> Lookup[b, "Domain", True],
  "RemainderDerivativeOrder" -> order|>];
```

This is a genuine improvement for independently constructed fixed operands: its source recurrence is $d_{n+1} = d_n + 1$. It is not a complete remedy for the normal scalar path. In a successful native source-substitution test, repeated `SeriesAdd[s, 0]` returned:

```
{{x, 2}, {x, 4}, {x, 8}, {x, 16}}
```

The repair changed the base of the exponential, not the nature of the scalar growth. The promoted scalar still brings a second copy of the current condition set into alignment.

The bundle retains `code/patch_merge_only.py` as an explicitly incomplete mechanism and regression counterexample. It should not be mistaken for the recommended fix. Recording this failed sufficiency test is important: a patch validated only with one fixed independent operand would appear successful while leaving the everyday scalar path defective.

4.2 Give condition intersection an idempotent representation

The stronger candidate makes two changes. First, alignment forms a duplicate-free conjunction of the already evaluated predicate expressions. Second, the final result keeps that aligned conjunction once.

```
seriesConditionClauses[c_And] :=
  Flatten[seriesConditionClauses /@ (List @@ c), 1];
seriesConditionClauses[True] := {};
seriesConditionClauses[c_] := {c};

seriesConditionJoin[a_, b_] := And @@ DeleteDuplicates[
  Join[seriesConditionClauses[a], seriesConditionClauses[b]]];

seriesAlign[a_, b_] := Join[
  b /. b["LogVariable"] -> a["LogVariable"],
  <| "Assumptions" -> seriesConditionJoin[
    a["Assumptions"], b["Assumptions"]],
    "Domain" -> seriesConditionJoin[
      Lookup[a, "Domain", True], Lookup[b, "Domain", True]]|>];
```

This helper deliberately does *less* than Boolean simplification. It descends only through top-level conjunctions, drops `True`, and removes structurally identical clauses. A disjunction is one opaque clause; it is not distributed. Distinct inequalities, parameter restrictions, quantifiers or other predicates are retained. The helper does not rewrite inside a held predicate or assert that two merely similar formulas are equivalent.

The stable first-occurrence order is useful for predictable output. It means the join need not be structurally commutative even though its truth condition is commutative. For example, the output order for $A \wedge B$ can differ from the output order for $B \wedge A$ without changing the admissible domain.

Proposition 4.1 (Semantic preservation). *For already evaluated Boolean predicates A and B , structural conjunction flattening, deletion of `True`, and removal of repeated identical clauses preserves $A \wedge B$. Replacing the old persisted $A \wedge (A \wedge B)$ by that normalized conjunction preserves the same truth condition.*

Proof. Flattening uses associativity of conjunction. Removing True uses its identity law. Removing an identical clause uses idempotence. No operation distributes through disjunction, changes a predicate's variables, weakens an inequality, or depends on a solver establishing an additional theorem. Applying these laws to the old persisted conjunction gives the new one. A retained False still makes the result false. $\square$

Corollary 4.2 (Bounded multiplicity under repeated neutral operations). *For the demonstrated paths with a fixed finite set of distinct conjunctive clauses, duplicate-free alignment followed by single ownership of its result retains at most one occurrence of each such clause. This applies both to an independent fixed operand and to a scalar representation that inherits the current condition set.*

This is a bound on repeated top-level clause occurrences, not a universal bound on predicate size. A single large disjunction remains large. Nor is a linear-time implementation of DeleteDuplicates assumed: exact structural comparison can have its own cost. The repair removes the exponential replication mechanism; it does not promise constant execution time for arbitrary condition formulas.

4.3 What remains unchanged

The candidate preserves seriesCompatible, the temporary proof environment, carrier alignment, jet addition and multiplication, real-coefficient validation, cutoff handling, derivative-order propagation, and refinement recipes. It does not strip conditions from a scalar merely because that scalar is zero or one. Instead it preserves the same conditions once. This avoids a second class of mistakes in which a neutral algebraic operation silently broadens the stated branch domain.

The choice to normalize in seriesAlign matters. Merely deduplicating the final printed domain would leave the internal representation and intermediate use of b inconsistent. Conversely, deleting the condition merge from alignment without checking its consumers would remove joint assumptions available while flattening the right representation. The two-site candidate retains those joint assumptions and gives their persistence a clear owner.

4.4 Executed candidate observations

Evidence status: Selected two-site source substitutions executed in Wolfram 15.0.0. Both old source fragments were found exactly once in the pinned standalone. After replacement and loading, the following results were obtained:

Operation chain	Domain copies, four steps	Finite expression
Add a fixed independent zero series	1, 1, 1, 1	x at every step
Add scalar zero	1, 1, 1, 1	x at every step
Multiply by scalar one	1, 1, 1, 1	x at every step

A distinct-parameter control combined ax under $a > 0$ with bx under $b > 0$. The result was $(a + b)x$, with Assumptions $\rightarrow a > 0 \ \&\& \ b > 0$ and TargetDomain $\rightarrow x > 0$. Combining incompatible parameter conditions $a > 0$ and $a < 0$ still returned Failure["IncompatibleDomains", ...].

Those observations validate the local mechanism on the stated native inputs. They do not validate every arithmetic family, every restored object or the complete modular integration. The Python tool stages equivalent expressions with explanatory comments and whitespace differences. The exact whole file emitted by that staging tool was not loaded as an integrated package in this audit.

5 Distinction from the existing review backlog

The nearest existing finding is P07, which concerns nested recipes, whole retained operands, cached state and provenance serialization. Report 1 A13 requests storage profiling; report 4 A04 gives an abstract repeated-self-multiplication recipe-tree recurrence and recommends a shared graph. Those observations are not reissued as new recommendations here.[5, 8, 9]

CG-01 differs in four concrete respects. Its measured quantity is the *current operational domain*, not a recipe tree or serialization size. Its fixed-operand witness does not repeatedly square a growing object. Its scalar witness traces an additional copy through ordinary operand promotion. Its repair changes condition joining while leaving the entire recipe representation in place. Even a perfectly shared recipe graph would not, by itself, prevent `seriesBinary` from constructing the repeated predicate list.

P08's broad suggestion to profile proof and normalization calls is also not a substitute for this repair. The defect is not merely that a theorem prover is slow on a hard condition; the input condition is repeatedly enlarged with logically redundant material. The earlier Mathics sign/realness recursion finding likewise has a different source mechanism and engine scope. CG-01 reproduces in Wolfram 15 without invoking that Mathics-only recursion.[5, 7]

The novelty ledger records these distinctions and its limitations. It also records rejected or duplicate candidates, without adding them to the article's findings count. In particular, the zero-order derivative cutoff issue and negative-source Gamma/Barnes power questions were excluded after matching prior reports. A suspected numerical source-domain bypass was refuted by following the public wrapper and running branch controls. The purpose of that ledger is to prevent a later intake from counting the same evidence twice, not to expand the issue total.

6 Controls and the boundaries of the evidence

6.1 Ordinary observable comparisons

A baseline screen used six functions,

$$1 + x + x^2, \quad e^x, \quad \frac{1}{1-x}, \quad \sin x, \quad \log(1+x), \quad \frac{x}{1+x},$$

and seven observables of a formal variable z ,

$$z^2, \quad z^{-1}, \quad \sqrt{z}, \quad e^z, \quad \sin z, \quad \cos z, \quad \log z.$$

The input expansions used package backend and cutoff four. For a returned ordinary series with numeric remainder power $0 < P < 20$, the comparison computed

$$\text{Normal}(\text{Series}[g(f(x)) - \text{Normal}(s_g), \{x, 0, \lceil P \rceil - 1\}]) .$$

This uses the returned `RemainderPower`, not an assumed property that might be absent. Forty-one of the 42 attempted combinations met the screen's admission test; none of those 41 produced a discrepancy. The original summarized response did not identify the one excluded combination, so the evidence file does not invent that information.

This screen is useful negative evidence, but not a proof of observable correctness. The oracle is native `Series`, and the screen is restricted to simple ordinary inputs. It was run on the unchanged baseline, not rerun as a complete screen after the candidate substitution. The supplied diagnostic runner records every combination explicitly and separates unassessed outputs and unavailable oracles from passes.

6.2 Source-domain controls on inverse numerical evidence

A source-level concern about an internal transformed-coordinate numerical helper disappeared when the public entry point was traced completely. The public `InverseNumericalCheck` applies a retained-source-domain check after the transformed helper returns.[10, 11]

Two useful native controls used a source restriction $x > 3$. For $f(x) = \exp(x^2)$, target e^4 corresponds to root two and was rejected as outside the branch; target e^{16} returned root four. For $f(x) = \exp(x^2 + x)$, the package used its `TargetLog` coordinate path: target e^6 was rejected, whereas e^{20} returned root four. The latter approximation error was about 4.228×10^{-9} at the tested cutoff and working precision.

These observations are retained as controls, not new defects or interval certificates. They also illustrate why inspecting a private helper without its public postconditions can produce a false positive. In the candidate work, the incompatible-domain control serves the same discipline: shrinking a condition expression is not acceptable if it stops rejecting contradictory input restrictions.

6.3 Independent tests

The final local Python run contains **26 passing unittest methods**. They cover exact recurrence models, semantic equivalence over finite Boolean assignments, scalar versus fixed-operand growth, opaque-clause treatment, and guarded transformation of transcribed source fixtures. Drift, duplicate definitions, reapplication and unrelated lookalike sites are tested separately.

These tests execute neither Wolfram Language nor Mathics. A Python model of conjunction is not an implementation test of a kernel's evaluator; a fixture replacement is not a full package build. Their value is narrower: they verify the reasoning model and that the patch tool refuses unexpected source shapes instead of silently performing a broad textual rewrite.

6.4 What was not obtained

No actual Mathics3 execution was available. No full upstream acceptance run, full staged modular integration, full staged standalone integration, replicated performance comparison or patched 41-case screen was completed. Several remote calls returned service or network errors; those failed calls are not counted as package failures. Only successful observations appear in the native ledger.

The distinction between a supplied regression script and an executed script is explicit throughout the bundle. The selected native mechanisms were exercised in submitted connector calls. The assembled `run_condition_probes.wl` and `run_observable_screen.wl` files are reproduction aids, not retrospectively manufactured run logs.

7 Wolfram 15 and Mathics3: targeted portability analysis

The reviewed loading path supplies a shared arithmetic implementation and isolates Mathics compatibility behavior in separate modules and contexts. Its compatibility layer includes association operations, scoped returns and bounded algebraic helpers; its numerical layer deliberately distinguishes unsupported high-precision reference solving from genuine numerical evidence. These are source-level observations, not a fresh claim that every public function is portable.[12, 13, 14]

CG-01 originates above those engine-specific details. The redundant conjunction is written in the ordinary shared binary engine, and scalar promotion also uses shared code. Thus the source concern applies to the Mathics path even though exact runtime multiplicities might differ because an evaluator or simplifier can canonicalize conditions differently.

The candidate helper uses ordinary structural operations: Head-based pattern dispatch through `And`, lists, one-level `Flatten`, `Join`, `DeleteDuplicates` and conjunction application. It does not require quantifier elimination, a native limit engine or an unimplemented numerical function. That keeps the portability obligation small, but it does not discharge it. Actual Mathics acceptance must still verify dispatch, exact structural comparison, preservation of distinct predicates, load/reload behavior and integration with the compatibility context.

7.1 Acceptance matrix

Obligation	Wolfram 15 evidence	Mathics3 evidence
Unchanged fixed-zero domain growth	Measured through eight operations	Not run
Unchanged scalar-zero domain growth	Measured through four operations	Not run
Merge-only repair remains exponential	Measured scalar counts 2, 4, 8, 16	Not run
Complete repair, three neutral chains	Selected mechanism passed four steps per chain	Not run
Distinct parameter conditions retained	Selected mechanism passed	Not run
Contradictory parameter conditions rejected	Selected mechanism passed	Not run
Full supplied condition runner	Not run as a complete program	Not run
Broad ordinary observable grid	41 eligible baseline cases, no discrepancy	Not run
Complete staged package integration	Not run	Not run

A release decision should use a matched version-pinned run on both engines rather than extrapolating the Wolfram observations. Failure of an unrelated unsupported feature should not be interpreted as evidence about this condition join; conversely, successful package loading is not sufficient acceptance of the repair.

8 Focused implementation and development plan

8.1 Integrate the small source repair first

Stage the complete candidate against efa1aeec, inspect the two-site diff, and validate the ordinary arithmetic path before widening the change. The fixture tool intentionally refuses changed definitions and existing candidate helper names. If the source has moved, reconcile the patch with the actual code rather than weakening those guards.

The maintained modular source is the proper integration point. A standalone candidate is convenient for isolated testing, but the normal repository generation process should produce the eventual distribution file from the accepted modular change. Editing only the generated artifact would leave the source of truth divergent. This is an integration instruction for the supplied repair, not a new general build-system finding.

8.2 Adopt an operational-condition invariant

The narrowly useful development addition is an invariant at this boundary: repeated intersection of the same accepted conjunctive predicate must not increase its persistent occurrence count. Keep this

invariant separate from the mathematical equality of finite expressions, remainder precision and refinement provenance.

Test scalar promotion explicitly. The failed merge-only candidate demonstrates that fixed independent operands alone are insufficient. Include fixed-left and fixed-right chains, scalar zero and one, nontrivial regular operands, and repeated shared domains. For each case compare expressions and remainder metadata in addition to counting predicate occurrences.

Distinct conditions require equally deliberate tests. Use different real parameter restrictions, incompatible restrictions, nested top-level conjunctions, duplicated disjunction clauses and formulas containing held subexpressions. The required behavior is exact duplicate removal, not a new logic solver or automatic weakening of branch predicates. A broad repository-wide replacement of every **And** would be unjustified.

8.3 Measure the repaired boundary, not an unrelated aggregate

The most diagnostic metric is the current domain's predicate multiplicity and leaf count. The full object size mixes operational metadata with histories, caches and other previously reviewed concerns. Keep those measurements separate so that unchanged recipe costs cannot conceal a successful condition repair, and recipe compaction cannot conceal a remaining condition defect.

For runtime measurements, compare identical package revisions apart from the candidate, the same neutral-operation chains, the same engine versions and the same evaluation settings. Use multiple fresh-session repetitions and report dispersion. Do not infer a speedup from the unreplicated baseline timing table in this article. An initially small exact expression is intentional: it minimizes confounding from coefficient growth.

This plan does not re-propose a shared provenance graph, global memoization, a new capability API, multivariate transseries or a general resource-budget redesign. Those are already part of the review baseline. The incremental development opportunity here is smaller and directly testable: make operational condition intersection idempotent in representation as well as in logic.

9 Reproducing and using the bundle

9.1 Contents and execution order

Path	Purpose
article/audit.tex, article/audit.pdf	Editable source and rendered article
code/patch_conditions.py	Recommended guarded two-site candidate staging
code/patch_merge_only.py	Incomplete first repair, retained as a counterexample
code/condition_join.wl	Human-readable helper and alignment excerpt
code/run_condition_probes.wl	Focused baseline/merge-only/candidate diagnostic runner
code/run_observable_screen.wl	Explicitly classified observable comparison grid
code/test_independent_models.py	Executed Python model and source-fixture tests
fixtures/ evidence/	Retrieved-source excerpts used to test guarded patching Native transcriptions, execution scope, coverage, novelty and Python test output
README.md, NOTICE.md, LICENSE	Instructions, evidence distinctions and MIT-0 notices

Run the independent tests from the bundle's code directory:

```
python test_independent_models.py
```

Stage a candidate into a new path; the tool will not overwrite the input or an existing destination:

```
python code/patch_conditions.py \  
/path/to/pinned/AsymptoticAnalysis.wl \  
/path/to/staged/AsymptoticAnalysis-candidate.wl
```

The same tool accepts the pinned modular `SeriesOperations.wl`. It expects the exact definition shapes present in this snapshot and is not a general Wolfram Language source parser.

In a fresh evaluator session, explicitly select the local file and expected mode:

```
$AuditPackage = "/absolute/path/to/AsymptoticAnalysis-candidate.wl";  
$AuditMode = "Candidate";  
Get["/absolute/path/to/bundle/code/run_condition_probes.wl"];
```

Use mode "Baseline" for unchanged source and "MergeOnly" only when intentionally testing the incomplete first repair. The runner's source comments distinguish native observations from additional source-derived expected cells. No exact timing or byte-count values are asserted as portable acceptance thresholds.

9.2 Evidence handling

The native observations JSON is a transcription of successful tool outputs. It is not a capture of a single continuously running kernel, and it does not contain a fabricated full-suite pass total. The callable evaluation environment was stateless, so each successful package probe loaded the pinned standalone afresh. The Python test output, by contrast, is an actual local captured run.

The archive contains no checksum files. Its exact revision identifier, source locations, transformation guards and evidence scope are sufficient to reproduce the intended review context without inventing a checksum manifest. Build instructions for the article are in the root README.

10 Final assessment

A neutral arithmetic operation should preserve not only the mathematical result but also a manageable representation of its unchanged validity conditions. At the reviewed revision, that operational contract fails: condition ownership is obscured by alignment, the left conditions are merged twice, and scalar promotion duplicates the current condition set again.

The result is unusually easy to isolate. The unchanged answer is the exact one-term function x . The observed current-domain counts follow the source-derived exponential recurrences exactly. The first plausible repair is demonstrably insufficient on scalar input. The stronger repair has a short Boolean correctness argument and successful focused Wolfram observations, including preservation and rejection controls for parameter domains.

That combination supports integrating and testing the local candidate. It does not justify relabeling existing backlog entries as new discoveries, declaring the entire repository correct, or claiming Mathics validation that did not occur. The useful deliverable is the specific new invariant, its public witnesses, its bounded repair mechanism, and an explicit path from the current focused evidence to two-engine acceptance.

References

- [1] VladimirReshetnikov/Asymptotic, reviewed commit. <https://github.com/VladimirReshetnikov/Asymptotic/commit/efalaeec4845a9c35e140963a0333d0c9ec33b05>. All repository references below are pinned to this revision.
- [2] `src/Kernel/SeriesOperations.wl`. Especially lines 93–121 (flattening and result construction), 123–131 (compatibility and alignment), and 133–151 (binary operations).
- [3] `src/Kernel/SeriesArithmetic.wl`. Especially lines 35–43 (public arithmetic), 49–69 (regular operand promotion), and 71–92 (binary dispatch).
- [4] Review index and retained/retired package baseline.
- [5] Maintained finding register. In particular the six-wave scope statement, P07 and P08.
- [6] Wave-5 review index.
- [7] Wave-6 review index.
- [8] Report 1, complete standalone article, A13 (provenance and retained-state storage policy).
- [9] Report 4, A04 (recipe-tree growth and sharing). Read as a predecessor, not as a new result of this audit.
- [10] Main modular kernel, public `InverseNumericalCheck` wrapper around lines 1275–1295.
- [11] `src/Kernel/CoordinateInverse.wl`, transformed-target construction and numerical evidence.
- [12] `src/Kernel/MathicsCompatibility.wl`, isolated compatibility context and structural helpers.
- [13] `src/Kernel/MathicsNumerical.wl`, precision-aware numerical dispatch.
- [14] Main modular kernel, final module-loading sequence and Mathics context cleanup, lines 1367 onward.